

SPRAWOZDANIE

Zajęcia: Grafika komputerowa
Prowadzący: mgr inż. Jakub Krawczyk

Laboratorium: 9

Temat: "Raymarch"

Jakub Kleczek
Informatyka I stopień,
stacjonarne,
4 semestr,
Gr.1b

1. Polecenie

Masz dwa kształty: koło i prostokąt. Oba zdefiniowane przez funkcje SDF w fragment shaderze, oba się animują — koło przesuwa się po jakiejś trajektorii, prostokąt się obraca albo zmienia proporcje, cokolwiek nietrywialnego.

Połącz je jedną z operacji:

- suma — $\min(a,b)$
- przecięcie — $\max(a,b)$
- różnica — $\max(a, -b)$

Aktywną operację kontroluje uniform (int 0/1/2), zmieniany trzema przyciskami w HTML. Wnętrze wynikowego kształtu ($SDF < 0$) ma inny kolor niż tło.

Funkcje SDF wykorzystaj z instrukcji

2. Wprowadzone dane (fragmenty)

```
float sdSphere(vec3 p, float r) {
    return length(p) - r;
}

float sdBox(vec3 p, vec3 b) {
    vec3 q = abs(p) - b;
    return length(max(q, vec3(0.0))) + min(max(q.x, max(q.y, q.z)), 0.0);
}

mat2 rotate2d(float a) {
    float s = sin(a);
    float c = cos(a);
    return mat2(c, -s, s, c);
}

float scene(vec2 uv) {
    vec2 circlePos = vec2(
        0.52 * sin(u_time * 0.85),
        0.28 * sin(u_time * 1.35)
    );
    float circle = sdSphere(vec3(uv - circlePos, 0.0), 0.38);

    vec2 boxPos = vec2(0.10 * sin(u_time * 0.55), -0.02);
    float angle = u_time * 0.75;
    vec2 boxUv = rotate2d(angle) * (uv - boxPos);
    vec3 halfSize = vec3(
        0.42 + 0.16 * sin(u_time * 1.10),
        0.23 + 0.10 * cos(u_time * 1.45),
        0.06
    );
    float box = sdBox(vec3(boxUv, 0.0), halfSize);

    if (u_operation == 0) {
        return min(circle, box);
    }

    if (u_operation == 1) {
        return max(circle, box);
    }

    return max(circle, -box);
}
```

```

const canvas = document.getElementById("gl");
const gl = canvas.getContext("webgl", { antialias: true });

if (!gl) {
  throw new Error("WebGL is not available in this browser.");
}

const vertexSource = document.getElementById("vertex-shader").textContent;
const fragmentSource = document.getElementById("fragment-shader").textContent;

function compileShader(type, source) {
  const shader = gl.createShader(type);
  gl.shaderSource(shader, source);
  gl.compileShader(shader);

  if (!gl.getShaderParameter(shader, gl.COMPILE_STATUS)) {
    const info = gl.getShaderInfoLog(shader);
    gl.deleteShader(shader);
    throw new Error(info);
  }

  return shader;
}

const program = gl.createProgram();
gl.attachShader(program, compileShader(gl.VERTEX_SHADER, vertexSource));
gl.attachShader(program, compileShader(gl.FRAGMENT_SHADER, fragmentSource));
gl.linkProgram(program);

if (!gl.getProgramParameter(program, gl.LINK_STATUS)) {
  throw new Error(gl.getProgramInfoLog(program));
}

gl.useProgram(program);

const vertices = new Float32Array([
  -1, -1,
  1, -1,
  -1, 1,
  1, 1,
  1, -1,
  1, 1
]);

const buffer = gl.createBuffer();
gl.bindBuffer(gl.ARRAY_BUFFER, buffer);
gl.bufferData(gl.ARRAY_BUFFER, vertices, gl.STATIC_DRAW);

const positionLocation = gl.getAttribLocation(program, "a_position");
gl.enableVertexAttributeArray(positionLocation);
gl.vertexAttribPointer(positionLocation, 2, gl.FLOAT, false, 0, 0);

const uniforms = {
  resolution: gl.getUniformLocation(program, "u_resolution"),
  time: gl.getUniformLocation(program, "u_time"),
  operation: gl.getUniformLocation(program, "u_operation")
};

let operation = 0;
const buttons = Array.from(document.querySelectorAll("[data-op]"));

buttons.forEach((button) => {
  button.addEventListener("click", () => {
    operation = Number(button.dataset.op);
    buttons.forEach((item) => {
      item.setAttribute("aria-pressed", String(item === button));
    });
  });
});

function resize() {
  const dpr = Math.min(window.devicePixelRatio || 1, 2);
  const width = Math.floor(canvas.clientWidth * dpr);
  const height = Math.floor(canvas.clientHeight * dpr);

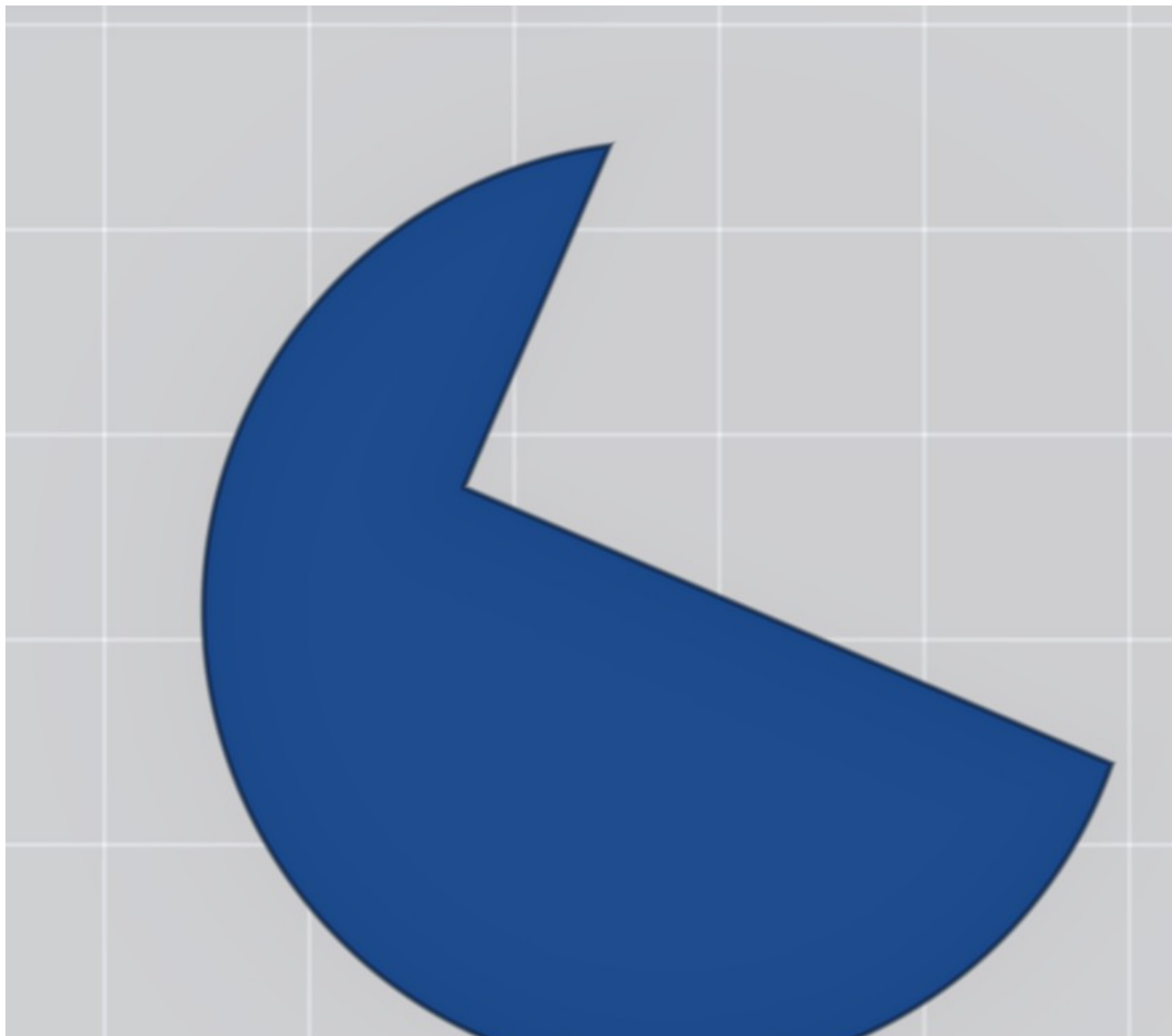
  if (canvas.width !== width || canvas.height !== height) {
    canvas.width = width;
    canvas.height = height;
    gl.viewport(0, 0, width, height);
  }
}

function render(now) {
  resize();
  gl.uniform2f(uniforms.resolution, canvas.width, canvas.height);
  gl.uniform1f(uniforms.time, now * 0.001);
  gl.uniform1i(uniforms.operation, operation);
  gl.drawArrays(gl.TRIANGLES, 0, 6);
  requestAnimationFrame(render);
}

requestAnimationFrame(render);

```

3. Wynik działania



4. Wykorzystane komendy:

<https://github.com/DziadekSwinka/Grafika/tree/872ba7d58c9a4b67a3925f2e21622b65b0abe453/Lab9%20-%20Raymarch>

5. Wnioski:

W ramach laboratorium zaimplementowano algorytm Raymarchingu w shaderze fragmentów, który odpowiada za renderowanie trójwymiarowej sceny z wykorzystaniem funkcji SDF (Signed Distance Fields). Aplikacja umożliwia dynamiczną animację sfery i prostopadłościanu oraz interaktywne modyfikowanie ich geometrii w czasie rzeczywistym poprzez operacje CSG sterowane z poziomu interfejsu HTML.